

---

**Agaro Campus**  
**School Of Computing**  
**Advanced Database System**  
**Lab**



# SQL Object Types

---

- ▶ **This section describes SQL object types including:**
  - ▶ *creating Objects types*
    - ▶ *Create, Insert, select ,delete*
  - ▶ *Constraints for Object Tables*
  - ▶ *Type evaluations*
  - ▶ *Collection types*
  - ▶ *Rules for REF Columns and Attributes*
  - ▶ **Object Methods**
  - ▶ **Types and Subtypes**

# Object Types

---

- ▶ An **object type** is a kind of data type.
- ▶ We can use it in the same ways as **number or varchar2, integer.**
- ▶ We use a variable of an object type to contain a value of that object type.
- ▶ A value of an object type is an instance of that type.
- ▶ An object instance is also called an object.

# CREATE TYPE

---

- ▶ **CREATE TYPE** statement is used to create the specification of an object type, a SQLJ object type, varray, a nested table type.
- ▶ You create object types with the **CREATE TYPE** and the **CREATE TYPE BODY** statements.
- ▶ The **CREATE TYPE** statement specifies the name of the object type, its attributes, methods, and other properties.
- ▶ The **CREATE TYPE BODY** statement contains the code for the methods that implement the type.

## *Examples*

---

► **Create OR Replace Type<Type\_Name> AS Object**  
**(<Member variables declarations>**  
**<Member function | procedure declarations>**  
**);**

**/**

► **Examples::**

**Create OR Replace Type PointType AS Object(**  
**x integer,**  
**y integer**  
**);**

**/**

# Con

---

► **Create table PointTab(  
    Pid varchar(10),  
    Pvalue pointType);**

*Now it's possible to apply OQL commands*

*Insert, select, update and delete, modify*

**Insert into pointTab1 values('P1',pointType(3,4));**

**Insert into pointTab1 values('P2',pointType(5,2));**

**Select \* from pointTab1;**

```
PID
-----
PVALUE(X, Y)
-----
p1
POINTTYPE1(3, 4)
p2
POINTTYPE1(5, 2)
```

# Notes

---

- ▶ For insert and update statement we must use the name of **user defined data type**.
- ▶ To update, select or delete an individual value from collections, we must use the name iterator of the user defined data type. **<iterator.column\_name>**

p.pvalue.x

p.pvalue.y

p.pid

Select p.\* from pointTab p;

Select p.pvalue.x from pointTab p;

# Con

---

- ▶ Select p.pvalue.y from pointTabl p;
- ▶ Select p.pvalue.x from pointTabl p;

```
SQL> select p.pvalue.y from pointtab1 p;
```

```
  PVALUE.Y
```

```
-----  
         4  
         2
```

```
SQL> select p.pvalue.x from pointtab1 p;
```

```
  PVALUE.X
```

```
-----  
         3  
         5
```

# Update commands

- Update pointTab1 p set p.pvalue.x =7 where p.pvalue.x=3;

```
SQL> update pointTab1 p set p.pvalue.x=7 where p.pvalue.x=3;
1 row updated.
```

```
SQL> select p.pvalue.x from pointtab1 p;
```

| PVALUE.X |
|----------|
| 7        |
| 5        |

```
SQL> update pointTab1 p set p.pvalue=pointType1(10,12) where p.pid='p1';
1 row updated.
```

```
SQL> select p.pvalue.x from pointtab1 p;
```

| PVALUE.X |
|----------|
| 10       |
| 5        |

```
SQL> select p.pvalue.y from pointtab1 p;
```

| PVALUE.Y |
|----------|
| 12       |
| 2        |

## Delete commands

---

- ▶ Delete from pointTabl p where p.pvalue.x=5;
- ▶ Delete from pointTabl p where p.pid='p2';

```
SQL> delete pointTabl p where p.pvalue.x=5;
1 row deleted.
SQL> select p.pvalue.x from pointtab1 p;
  PVALUE.X
-----
        10
```

# Example2

```
SQL> create or replace type pointT1 as object(  
  2  x integer;  
  3  y integer);  
  4  /
```

Type created.

```
SQL> create or replace type LineT1 as object(  
  2  p1 pointT1,  
  3  p2 pointT1  
  4  );  
  5  /
```

Type created.

```
SQL> create table LineTab1 (  
  2  lineId varchar(3),  
  3  lineDetails LineT1);
```

Table created.

```
SQL> insert into LineTab1 values('L1', lineT1(pointT1(2,4),pointT1(3,6)));
```

1 row created.

```
SQL> insert into LineTab1 values('&lineid', lineT1(pointT1(&xvalue,&yvalue),pointT1(&x,&y)));
```

Enter value for lineid: 12

Enter value for xvalue: 11

Enter value for yvalue: 12

Enter value for x: 13

Enter value for y: 14

old 1: insert into LineTab1 values('&lineid', lineT1(pointT1(&xvalue,&yvalue),pointT1(&x,&y)))

new 1: insert into LineTab1 values('12', lineT1(pointT1(11,12),pointT1(13,14)))

1 row created.

# Con

```
SQL> select l.lineDetails.p1.x,l.linedetails.p2.x from linetab1 l;
```

| LINEDETAILS.P1.X | LINEDETAILS.P2.X |
|------------------|------------------|
| 2                | 3                |
| 11               | 13               |

```
SQL> update linetab1 l set l.linedetails=lineT1(pointT1(40,50),pointT1(60,80)) where l.linedetails.p1.x=2;
```

1 row updated.

```
SQL> select p.* from lineTab1 p;
```

LIN

---

LINEDETAILS(P1(X, Y), P2(X, Y))

-----

L1

LINE1(POINTT1(40, 50), POINTT1(60, 80))

L2

LINE1(POINTT1(11, 12), POINTT1(13, 14))

# Con

---

```
SQL> update linetab1 l set l.linedetails=lineT1(pointT1(40,50),pointT1(60,80)) where l.lineid='12';  
1 row updated.
```

```
SQL> select l.linedetails.p1.x,l.linedetails.p2.x from linetab1 l where l.lineid='12';
```

| LINEDETAILS.P1.X | LINEDETAILS.P2.X |
|------------------|------------------|
| 40               | 60               |

```
SQL> select l.linedetails from linetab1 l where l.lineid='12';
```

| LINEDETAILS(P1(X, Y), P2(X, Y))          |
|------------------------------------------|
| LINET1(POINTT1(40, 50), POINTT1(60, 80)) |

# Object Table

---

- ▶ *If a table is completely made up of an object type (class) then it is called an **object table**.*
- ▶ *All the records in the object table are called **row object**.*
- ▶ ***Example***

```
Create or replace type customer_typ as object(  
  Custid varchar(5),  
  Custname varchar(12),  
  Custphone number(10));  
/  
Create table custTab of customer_typ;  
Insert into custTab values(customer_typ('c1','xxx',0911));
```

## Collection types

---

- ▶ ***Collections*** are used to model multi-valued attributes and many to many relationships.

**Two possibilities to make list with in oracle,  
those are**

- ▶ **Varrays**
- ▶ **Nested tables**

# Varrays

---

- ▶ A **varray** is an ordered set of data elements.
  - ▶ All elements of a given varray are of the same datatype or a subtype of the declared one.
  - ▶ They treated as the **collections** of values as an **atomic unit**
  - ▶ We can not **delete, access or update** a single element in collection.

# Varray

---

- ▶ *To define a varray, we specify the maximum number of elements it can contain.*
- ▶ *The number of elements in an array is the **size of the array**.*
- ▶ *Oracle allows arrays to be of variable size, which is why they are called varrays.*

# Varray is used

---

- ▶ To store only a fixed number of items,
- ▶ To loop through the elements in order,
- ▶ To retrieve and manipulate the entire collection as a value.
- ▶ *Syntax*
- ▶ *Create or Replace type **Varray\_name** AS **VARRAY**(**n**) OF datatype; /*

*Example:*

*Create or replace type email\_typ as varray(3) of  
varchar(20);*

*/*

## Example 2

---

*Create or replace type phone\_typ as object (phone  
number(10));*

*/*

***CREATE TYPE phone\_list AS VARRAY(5) OF  
phone\_typ; /***

*/*

**Create table stud(  
stid number(4),  
stname varchar(12),  
Stphone phone\_list );**

# Insert records

---

- ▶ *Insert into stud values(10,'xxx', phone\_list  
AS 10 (phone\_typ(0910), phone\_typ(0911),phone\_typ(0912),  
phone\_typ(0913),phone\_typ(0914)));*
- ▶ *Insert into stud values(11,'yyy', phone\_list  
(phone\_typ(09102)));*
- ▶ *Insert into stud values(12,'zzz', phone\_list  
(phone\_typ(09102),phone\_typ(091342)));*
- ▶ *Update stud set  
stphone=phone\_list(phone\_typ(**091234**),phone\_typ(0913  
42)) where stdid=12;*

# Nested Tables

---

- ▶ A ***nested table*** is an unordered set of data elements, all of the same datatype.
  - ▶ Can have any number of items
  - ▶ No upper limit
- steps*
1. Create object type
  2. Create a nested list using table keyword
  3. Implement the nested list in the nested table

# Example

---

- ▶ **Create or replace type phone\_type as object(  
phone number(10));  
/**
- ▶ **Create or replace type phone\_nests as table of  
phone\_type;  
/**
- ▶ **Create table customers\_phone(  
Cusid integer,  
Custname varchar(12),  
Custphone phone\_nests) nested table Custphone store as  
phone\_num ;**

- ▶ Insert into customers\_phone values(1,'dawod',  
phone\_nests(phone\_type(0911), phone\_type(0913),  
phone\_type(0917)));
  - ▶ Insert into customers\_phone values(2,sol',  
phone\_nests(phone\_type(0911), phone\_type(0913)));
  - ▶ SQL> select \* from customers\_phone;
  - ▶ SQL> select s.custphone from customers\_phone s;
- CUSTPHONE(PHONE)

-----

PHONE\_NESTS(PHONE\_TYPE(911), PHONE\_TYPE(913),  
PHONE\_TYPE(917))

PHONE\_NESTS(PHONE\_TYPE(911), PHONE\_TYPE(913))

- ▶ *Select **i.\*** from table(select p.custphone from customers\_phone p where p.cusid=1)i;*
- ▶ ▶ *insert into table(select p.custphone from customers\_phone p where p.cusid=1) values(phone\_type(0945));*
- ▶ *Update table(select p.custphone from customers\_phone p where p.cusid=1)i set value(i)=phone\_type(0977)where Value(i)=phone\_type(0911);*
- ▶ *Delete from table(select p.custphone from customers\_phone p where p.cusid=1)i where value(i)= phone\_type(0977);*

## Example Two

```
SQL> connect system/123
```

Connected.

```
SQL> create or replace type productsTyp as object(  
  2  item varchar(20));  
  3  /
```

Type created.

```
SQL> create or replace type prod_List as table of productsTyp;  
  2  /
```

Type created.

```
SQL> create table product(  
  2  prodid integer,  
  3  pqty integer,  
  4  price integer,  
  5  pname varchar(12),  
  6  item_det prod_List)nested table item_det store as category;
```

Table created.

# Object Methods

---

- ▶ **The topics described in this section are:**
  - ▶ **Member Methods**
  - ▶ **Methods for Comparing Objects**
    - **Map Methods**
    - **Order Methods**
  - ▶ **Static Methods**
  - ▶ **Constructor Methods**

# Methods

---

- ▶ **Methods** are functions or procedures that can declare in an object type definition to implement behavior of objects type.
- ▶ **Member Methods**  
**Member** methods are used for manipulating the attributes of the object.
- ▶ You provide the declaration of a member method while declaring the object type.

- 
- ▶ *The object body defines the code for the member methods.*
    - ▶ *The object body is created using the **CREATE TYPE BODY** statement.*
  - ▶ ***Constructors** are functions that return a new object as its value.*
    - ▶ *Every object has a system defined constructor method.*
    - ▶ *The **name of the constructor** is same as the object type.*

```
residence := address('103A', 'M.G.Road', 'Jaipur', 'Rajasthan', '201301');
```

## ***Example1:***

---

***Create or replace type rectangle as object(  
rid integer,  
Length integer,  
Width integer,  
Map member function area return integer,  
Member procedure display);  
/***

**Create or replace type body rectangle as map member function area return integer**

**Is**

**Begin**

**Return length\* width;**

**End;**

**Member procedure display**

**Is**

**Begin**

**dbms\_output.put\_line('Rec\_id is' || rid);**

**dbms\_output.put\_line('Rec\_len is' || length);**

**dbms\_output.put\_line('Rec\_wid is' || width);**

**End;**

**End;**

Con...

---

*Create table rect1 (*

*Rect\_desc varchar(12),*

*Rect\_details rectangle);*

*Create table rect2 of rectangle;*

*Insert into rect1 values('small', rectangle(1,3,5));*

*Insert into rect2 values(2,5,8);*

*Select r.area() from rect2 r where r.rid=2;*

*Select r.rect\_details.area() from rect1 r where r.rid=1;*

Con...

---

***Declare***

***m rectangle;***

***i integer;***

***Begin***

***i:=&i;***

***Select value(c) into m from rect2 c where c.rid=i;***

***m.display;***

***End;***

***/***

Con...

---

***Declare***

***m rectangle;***

***i integer;***

***Begin***

***i:=&i;***

***Select r.rect\_details into m from rect1 r where  
r.rect\_details.rid=i;***

***m.display;***

***End;***

***/***

- Create or replace type squares as object (length integer;

Member Function squareArea return integer);

/

Create or replace type body squares as member function squareArea return integer

Is

area integer;

Begin

area :=length\*length;

Return area ;

End;

End;

/

Create table sqr of squares ;

Insert into sqr values(4);

Select c.squareArea () from sqr c;

# comparison methods

---

- ▶ *The comparison methods are used for comparing objects.*

*There are two ways to compare objects:*

- ▶ ***Map method:** The Map method is a function implemented in such a way that its value depends upon the value of the attributes.*
- ▶ ***Example** , for a customer object, if the customer code is same for two customers, both customers could be the same and one. So the relationship between these two objects would depend upon the value of code.*

## comparison methods

---

- ▶ **Order method:** *The Order methods implement some internal logic for comparing two objects.*
- ▶ *For example, for a rectangle object, a rectangle is bigger than another rectangle if both its sides are bigger.*

# Examples

```
SQL> create or replace type person_typ as object(  
  2  pid integer,  
  3  pname varchar(10),  
  4  pphone number(10),  
  5  pemail varchar(20),  
  6  map member function get_pid return integer,  
  7  member procedure display(self in out nocopy person_typ));  
  8  /
```

Type created.

```
SQL> create or replace type body person_typ as map member function get_pid return integer  
  2  is  
  3  begin  
  4  return pid;  
  5  end;  
  6  member procedure display(self in out nocopy person_typ)  
  7  is  
  8  begin  
  9  dbms_output.put_line('Per id is'||pid);  
 10  dbms_output.put_line('Per name is'||pname);  
 11  dbms_output.put_line('Per phone is'||pphone);  
 12  dbms_output.put_line('Per email is'||pemail);  
 13  end;  
 14  end;  
 15  /
```

Type body created.

```
SQL> create table contacts of person_typ;
```

```
Table created.
```

```
SQL> insert into contacts values(14,'Bob',090946,'bobmar@yahoo.com');
```

```
1 row created.
```

```
SQL> select c.get_pid() from contacts c;
```

```
C.GET_PID()
```

```
-----  
14
```

```
SQL> declare  
2  con person_typ;  
3  begin  
4  select value(c) into con from contacts c;  
5  con.display;  
6  end;  
7  /
```

```
Per id is14
```

```
Per name isBob
```

```
Per phone is90946
```

```
Per email isbobmar@yahoo.com
```

```
PL/SQL procedure successfully completed.
```

# Type Evolution

---

- ▶ Changing a object type is called type evolution.
  - ▶ You can make the following changes to an object type:
    - ✓ Add and drop attributes
    - ✓ Add and drop methods
    - ✓ **Modify** a numeric attribute to increase its length, precision, or scale
    - ✓ **Modify** a varying length character attribute to increase its length
    - ✓ Change a type's **FINAL** and **INSTANTIABLE** properties
    - ✓ Modify limit and size of **VARRAYs**
    - ✓ Modify length, precision, and scale of collection elements

# Example

---

*Create or replace type person\_type as object(*

*Pid integer,*

*pname varchar(12);*

*/*

*Alter type person\_type **add** attribute (phone number(10))  
cascade not including table data;*

*Alter type person\_type **drop** attribute pname cascade not  
including table data;*

*Alter type person\_type **modify** attribute (addr  
varchar(30)) cascade not including table data;*

***Modify is used only to increase the length of an attribute.***

# Con...

---

**Create table person of person\_type;**

**Alter table person upgrade including data;**

**Alter type person\_type add member function get\_phone  
return number cascade not including table data;;**

**Alter type person\_type add member procedure display  
cascade not including table data;;**

**Alter type person\_type drop member function get\_phone;**

# Type evaluation in collections

---

**Create or replace type email as varray(2) of  
varchar(20);**

**/**

**Alter type email modify element type varchar(50)  
cascade;**

**Alter type email modify limit 6 cascade;**

**Create or replace type email\_nest as table of  
varchar(10);**

**/**

**Alter type email\_nest modify element type  
varchar(25) cascade;**

- ▶ The **CASCADE** option for **ALTER TYPE** propagates a type change to dependent types and tables.
- ▶ **CASCADE** itself has options that let you choose whether to convert table data to the new type format as part of the propagation:
  - ▶ the option **INCLUDING TABLE DATA** converts the data;
  - ▶ the option **NOT INCLUDING TABLE DATA** does not convert it.

- ▶ By default, the **CASCADE** option converts the data.
  - ▶ In any case, table data is always returned in the format of the latest type version.
  - ▶ You can use the **INVALIDATE** option to drop a method that has been redefined, but the redefined versions in the subtypes must still be dropped manually.
  - ▶ The subtypes will remain in an invalid state until they are explicitly altered to drop the redefined versions.
  - ▶ Until then, an attempt to recompile the subtypes for revalidation will produce the error Method does not override

## ► Modifying the FINAL or INSTANTIABLE Property

- Altering an object type from **INSTANTIABLE** to **NOT INSTANTIABLE** is allowed only if the type has no table dependents.

- Altering an object type from **NOT INSTANTIABLE** to **INSTANTIABLE** is allowed anytime.

► This change does not affect tables.

- Altering an object type from **NOT FINAL** to **FINAL** is allowed only if the target type has no subtypes.

- altering an object type from **FINAL** to **NOT FINAL** or vice versa, you must use **CASCADE** to convert data in dependent columns and tables immediately. You may not use the **CASCADE option NOT INCLUDING TABLE DATA** to defer converting data.

- ▶ If you alter a type from **NOT FINAL** to **FINAL**, you must use **CASCADE INCLUDING TABLE DATA**.
- ▶ If you alter a type from **FINAL** to **NOT FINAL**, you may use either **CASCADE INCLUDING TABLE DATA** or **CASCADE CONVERT TO SUBSTITUTABLE**.

When you alter a type from **FINAL** to **NOT FINAL**, the **CASCADE** option you should choose depends on whether you want to be able to insert new subtypes of the type you are altering in existing columns and tables.



# Con...

---

- ▶ **By default, altering a type from FINAL to NOT FINAL enables you to create new substitutable tables and columns of that type, but it does not automatically make existing columns (or object tables) of that type substitutable. In fact, just the opposite happens: existing columns and tables of the type are marked NOT SUBSTITUTABLE AT ALL LEVELS.**

- ▶ If any embedded attribute of such a column is substitutable, an error is generated.
- ▶ New subtypes of the altered type cannot be inserted in such preexisting columns and tables.

To alter an object type to **NOT FINAL** in such a way as to make existing columns and tables of the type substitutable (assuming that they are not marked **NOT SUBSTITUTABLE**),

- ▶ use the **CASCADE** option **CONVERT TO SUBSTITUTABLE**.

# ALTER TYPE Statement for Type Evolution

## ▶ ALTER TYPE Options for Type Evolution

|                      |                                                                                                                                                                                                                                                                                           |
|----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| INVALIDATE           | Invalidates all dependent objects. Table data cannot be accessed again until it is validated; if it cannot be validated, it remains inaccessible.                                                                                                                                         |
| CASCADE              | Propagates the type change to dependent types and tables.                                                                                                                                                                                                                                 |
| INCLUDING TABLE DATA | Converts data stored in all user-defined columns to the most recent version of the column's type.<br>New attribute is added to the data and is initialized to NULL.<br>All tablespaces containing the table's data must be in read write mode; otherwise, the statement will not succeed. |
| FORCE                | Forces the system to ignore errors from dependent tables and indexes.                                                                                                                                                                                                                     |



# Con...

---

## NOT INCLUDING TABLE DATA

Leaves column data as is, associated with the current type version.

If an attribute is dropped from a type referenced by a table, then the corresponding column of the dropped attribute is not removed from the table.

## CONVERT TO SUBSTITUTABLE

Use when altering a type from FINAL to NOT FINAL:  
Converts data stored in all user-defined columns to the most recent version of the column's type and then marks these existing columns and object tables of the type  
**SUBSTITUTABLE AT ALL LEVELS.**



# Examples

---

```
SQL> create or replace type persontyp as object(
```

```
  2  pid integer,
```

```
  3  pname varchar(10));
```

```
  4  /
```

Type created.

```
SQL> alter type persontyp add attribute (phone number(10)) cascade not including table data;
```

Type altered.

```
SQL> alter type persontyp add attribute (addr varchar(10)) cascade not including table data;
```

Type altered.

```
SQL> desc persontyp;
```

| Name  | Null? Type   |
|-------|--------------|
| ----- |              |
| PID   | NUMBER(38)   |
| PNAME | VARCHAR2(10) |
| PHONE | NUMBER(10)   |
| ADDR  | VARCHAR2(10) |

---



# Con..

---

```
SQL> alter type persontyp drop attribute addr cascade not including table data;
```

Type altered.

```
SQL> alter type persontyp add member function getld return integer cascade not including table data;
```

Type altered.

```
SQL> alter type persontyp drop member function getld return integer cascade not including table data;
```

Type altered.

```
SQL> create table person of persontyp;
```

Table created.

```
SQL> alter table person upgrade including data;
```

Table altered.



```
SQL> create or replace type email as varray(2) of varchar(10);
```

---

```
/
```

Type created.

```
SQL> alter type email modify element type varchar(30) cascade;
```

Type altered.

```
SQL> alter type email modify limit 5 cascade;
```

Type altered.

```
SQL> create or replace type emailnest as table of varchar(10);
```

```
/
```

Type created.

```
SQL> alter type emailnest modify element type varchar(40) cascade;
```

Type altered.

---



# Constraints

---

- ▶ **Not null**
- ▶ **Primary key**
- ▶ **Unique**
- ▶ **Check**
- ▶ **Foreign key in ODB reference key(ref)**

# Examples

---

SQL> create or replace type stud as object(

2 sid integer,

3 sname varchar(12),

4 age integer,

5 phone number(10)

6 );

7 /

Type created.

SQL> create table new\_stud of stud(sid primary key);

Table created.

SQL> create table gradstud(

2 stdetail stud,

3 dname varchar(12),

4 class integer,

5 constraint c1 unique(stdetail.phone),

6 constraint c2 check(stdetail.age>20),

7 constraint c3 check(stdetail.sname is not null)

8 );

---

▶ Table created.

# References

---

- ▶ **A REF is a logical pointer to a row object that is constructed from the object identifier (OID) of the referenced object.**
  - ▶ It is an Oracle built-in datatype.
  - ▶ Many -to-one relationships, thus reducing the need for foreign keys.
- ▶ **REFs provide an easy mechanism for navigating between objects.**

# Example

---

- ▶ **CREATE TYPE emp\_person\_typ AS OBJECT (  
name VARCHAR2(30),  
manager REF emp\_person\_typ );  
/  
CREATE TABLE emp\_person\_obj\_table OF  
emp\_person\_typ;**
- ▶ **INSERT INTO emp\_person\_obj\_table VALUES (  
emp\_person\_typ ('John Smith', NULL));**
- ▶ **INSERT INTO emp\_person\_obj\_table  
SELECT emp\_person\_typ ('Bob Jones', REF(e))  
FROM emp\_person\_obj\_table e  
WHERE e.name = 'John Smith';**

# Ref

```
SQL> create or replace type emp as object(
```

```
2  eid integer,
```

```
3  ename varchar(12),
```

```
4  hod ref emp
```

```
5 );
```

```
6 /
```

Type created.

```
SQL> create table empl of emp;
```

Table created.

```
SQL> insert into empl values(1,'Sara',null);
```

1 row created.

```
SQL> insert into empl select emp(2,'semira',ref(e)) from empl e where e.ename='Sara';
```

1 row created.

```
SQL> insert into empl select emp(3,'dawod',ref(e)) from empl e where e.ename='Sara';
```

1 row created.

```
SQL> select * from empl;
```

| EID | ENAME | HOD |
|-----|-------|-----|
|-----|-------|-----|

|   |      |  |
|---|------|--|
| 1 | Sara |  |
|---|------|--|

|   |        |                  |
|---|--------|------------------|
| 2 | semira | 000022020891D... |
|---|--------|------------------|

|   |       |                   |
|---|-------|-------------------|
| 3 | dawod | 000022020891D3... |
|---|-------|-------------------|

# Inheritance in Object Type

---

- ▶ *Inheritance property allows the sub-object type to access all the attribute and members of the super object type or parent object type.*
- ▶ *The sub-object type is called inherited object type, and the super object type is called parent object type.*

# Types and Subtypes

---

- ▶ A **subtype** can be derived from a supertype either directly, or indirectly through intervening levels of other subtypes.
- ▶ subtype can directly derive only from a single supertype.
- ▶ It cannot derive jointly from more than one.
- ▶ A supertype can have multiple sibling subtypes, but a subtype can have at most one direct parent supertype.

- 
- ▶ **In other words, Oracle supports only single inheritance, not multiple inheritance.**
  - ▶ **A subtype is derived from a supertype by defining a specialized variant of the supertype.**
  - ▶ ***The below syntax shows the how to create parent and inherited object type.***
  - ▶ **Example shape type**

**Syntax:**

```
CREATE TYPE <object_type_name_parent> AS OBJECT  
(  
  <attribute_1> <datatype>,  
  .  
  .  
) NOT FINAL;  
/
```

SUPER

It implies that  
this can have sub

**Syntax:**

```
CREATE TYPE <object_type_name_sub> UNDER <object_type_name_parent>  
(  
  <attribute_1> <datatype>,  
  .  
  .  
);  
/
```

It implies that  
it is a sub

SUB Type

# Examples

---

► **Create or Replace type shape\_type as object(  
    shape\_id integer,  
    shapeName varchar(12),  
    Member function area return integer) Not final;  
    /  
create or replace type circle\_type **under** shape\_type(  
    radius integer,  
    overriding member function area return integer)  
    Not final;  
    /**

Con...

---

*Create or replace type sphere\_type under circle\_type(  
height integer,  
member function volume return integer);*

*/*

*Create or replace type body circle as overriding member  
function area return integer*

*Is*

*Begin*

*Return radius\*radius\*3.14;*

*End;*

*End;*

*/*

# Con...

---

**Create table shape of shape\_type;**

**Insert into shape values(shape\_type(1,'shape1'));**

**Insert into shape values(circle\_type(2,'circle',4));**

**Insert into shape values(sphere\_type(3,'sphere',4,4));**

**Select \* from shape;**

**Select value(c) from shape c where value (c) is of (circle\_type);**

**Select value(c) from shape c where value (c) is of (only  
circle\_type);**

**Select value(c).area() from shape c where value (c) is of (only  
circle\_type);**

# Steps

---

- ▶ ***We are going to execute the above program in the following steps***
  - ▶ ***Step1: Create SUPER type.***
  - ▶ ***Step2: Create SUB type and body.***
  - ▶ ***Step3: Creating an anonymous block to call the SUB type.***

# Examples

---

```
SQL> create or replace type emp_object as object(  
    emp_no number,  
    emp_name varchar2(12),  
    salary number,  
    manager number,  
    constructor function emp_object(p_emp_no number,  
    p_emp_name varchar,p_salary number ) return self as result,  
    member procedure insert_records,  
    member procedure display_records) not final;  
/  

```

Type created.

## ***Step2: Create SUB type and body.***

---

```
SQL> create or replace type sub_emp under emp_object(  
  2  default_manager number, member procedure insert_manager  
  );  
  3 /
```

Type created.

```
SQL> create table emp(  
  2  emp_no number(10),  
  3  emp_name varchar(12),  
  4  salary number(10,3),  
  5  manager number(10));
```

Table created.

### ***Step3: Creating an anonymous block to call the SUB type.***

---

```
SQL> create or replace type body sub_emp as member  
  procedure insert_manager  
  2  is  
  3  begin  
  4  insert into emp  
    values(emp_no,emp_name,salary,manager);  
  5  end;  
  6  end;  
  7  /
```

Type body created.

---

SQL> declare

2 emp\_desc sub\_emp;

3 begin

4 emp\_desc:=sub\_emp(12,'Dawod',7200,10,5);

5 emp\_desc.insert\_manager;

6 commit;

7 end;

8 /

PL/SQL procedure successfully completed.